

Lecture 06

2026-04-16

Welcome Back

Anonymous Activity 1

QR Code (it's getting better)

This is anonymous & optional, but helpful to Stats 20,



menti.com
9907 3567

or here is a [direct link](#)

The List

What is a list?

- We have seen vectors and matrices (only one type of data)
- We have seen data frames (can store multiple types)
- Now the list

```
is.list(rivers) # no this a vector
```

```
[1] FALSE
```

```
is.list(iris)
```

```
[1] TRUE
```

- iris is a data frame but all data frames are lists

```
is(iris)
```

```
[1] "data.frame" "list"      "oldClass"   "vector"
```

List

From the original documentation

Lists are another kind of data storage. Lists have elements, each of which can contain **any type of R object**, i.e. the elements of a list do not have to be of the same type. List elements are accessed through three different indexing operations. These are explained in detail in Indexing.

Lists are called “generic vectors”, and the basic vector types (e.g., integer, numeric, character etc.) that we have been using are referred to as “atomic vectors”.

Recall

- The data type in our simple vector must be all the same (all numeric, all character, all logical)
- If there is mixing, we say that R “coerces” the vector to be one type.
- But if data are stored in a list, they can maintain their own type.

Examples

Recall, an .RData file is a saved environment or a saved workspace. It can contain multiple items

```
load("state_database.RData")  
ls()
```

```
[1] "euro"          "iris"          "rivers"        "state_data"  
[5] "state.abb"     "state.area"    "state.center"  "state.division"  
[9] "state.name"    "state.region"  "state.x77"
```

Let's examine the list called `state_data`

```
str(state_data)
```

List of 10

```
$ name      : chr [1:50] "Alabama" "Alaska" "Arizona" "Arkansas" ...
$ abbrev    : chr [1:50] "AL" "AK" "AZ" "AR" ...
$ state_stats: num [1:50, 1:8] 3615 365 2212 2110 21198 ...
..- attr(*, "dimnames")=List of 2
.. ..$ : chr [1:50] "Alabama" "Alaska" "Arizona" "Arkansas" ...
.. ..$ : chr [1:8] "Population" "Income" "Illiteracy" "Life Exp" ...
$ region    : Factor w/ 4 levels "Northeast","South",...: 2 4 4 2 4 4 1 2 2 2 ...
$ division  : Factor w/ 9 levels "New England",...: 4 9 8 5 9 8 1 3 3 3 ...
$ center    : 'data.frame': 50 obs. of 2 variables:
..$ x: num [1:50] -86.8 -127.2 -111.6 -92.3 -119.8 ...
..$ y: num [1:50] 32.6 49.2 34.2 34.7 36.5 ...
$ area      : num [1:50] 51609 589757 113909 53104 158693 ...
$ euro      : Named num [1:11] 13.76 40.34 1.96 166.39 5.95 ...
..- attr(*, "names")= chr [1:11] "ATS" "BEF" "DEM" "ESP" ...
$ rivers    : num [1:141] 735 320 325 392 524 ...
$ iris      : 'data.frame': 150 obs. of 5 variables:
..$ Sepal.Length: num [1:150] 5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
..$ Sepal.Width : num [1:150] 3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
..$ Petal.Length: num [1:150] 1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
..$ Petal.Width : num [1:150] 0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
..$ Species     : Factor w/ 3 levels "setosa","versicolor",...: 1 1 1 1 1 1 1 1 1 1 ...
```

What the load() should reveal

The screenshot shows the RStudio interface for a project named 'my_stats_20'. The script editor on the left contains R code for loading a dataset and generating a QR code. The console at the bottom shows the command `load("state_database.RData")` being executed. The environment pane on the right displays the loaded data objects.

Script Editor:

```
29 - attribution
30 pdf:
31   geometry: "landscape"
32 execute:
33   echo: true
34   warning: true
35   message: false
36 ---
37
38 - # Welcome Back
39
40 - ## Anonymous Activity 1
41 - ### QR Code (it's getting better)
42
48:1 QR Code (it's getting better) >
```

Console:

```
> load("state_database.RData")
>
```

Environment Pane:

Object	Details
iris	150 obs. of 5 variables
state_data	List of 10
state.center	50 obs. of 2 variables
state.x77	num [1:50, 1:8] 3615 365 2212 2110 21198 ...

Files Pane:

Name	Size	Modified
pokemon.RData	40 KB	Apr 9, 2026, 6:22 AM
Project01_quarto.html	3.1 MB	Apr 9, 2026, 5:42 AM
Project01_quarto.pdf	1.2 MB	Apr 9, 2026, 5:43 AM
Project01_quarto.qmd	3.7 KB	Apr 9, 2026, 6:06 AM
Project01_RMarkdown.html	2.5 MB	Apr 9, 2026, 5:43 AM
Project01_RMarkdown.log	30.7 KB	Apr 9, 2026, 5:43 AM
Project01_RMarkdown.pdf	1.5 MB	Apr 9, 2026, 5:43 AM
Project01_RMarkdown.Rmd	4.3 KB	Apr 9, 2026, 6:08 AM
renv		
renv.lock	134.7 KB	Apr 9, 2026, 10:40 AM
spotify_songs.RData	2.7 MB	Apr 2, 2026, 1:49 AM
starwars_dataverse		
starwars.RData	4.4 KB	Apr 7, 2026, 2:12 AM
state_database.RData	5.1 KB	Apr 16, 2026, 9:55 AM

How to index (extract) elements from a list

- If you thought to use your tools (`$`, `[]`) from a data frame you have great instincts, a list offers us many possibilities:

```
state_data$abbrev ## must be a named element
```

```
[1] "AL" "AK" "AZ" "AR" "CA" "CO" "CT" "DE" "FL" "GA" "HI" "ID" "IL" "IN" "IA"  
[16] "KS" "KY" "LA" "ME" "MD" "MA" "MI" "MN" "MS" "MO" "MT" "NE" "NV" "NH" "NJ"  
[31] "NM" "NY" "NC" "ND" "OH" "OK" "OR" "PA" "RI" "SC" "SD" "TN" "TX" "UT" "VT"  
[46] "VA" "WA" "WV" "WI" "WY"
```

```
state_data[2] ## why only one value and not [r,c]
```

```
$abbrev
```

```
[1] "AL" "AK" "AZ" "AR" "CA" "CO" "CT" "DE" "FL" "GA" "HI" "ID" "IL" "IN" "IA"  
[16] "KS" "KY" "LA" "ME" "MD" "MA" "MI" "MN" "MS" "MO" "MT" "NE" "NV" "NH" "NJ"  
[31] "NM" "NY" "NC" "ND" "OH" "OK" "OR" "PA" "RI" "SC" "SD" "TN" "TX" "UT" "VT"  
[46] "VA" "WA" "WV" "WI" "WY"
```

```
state_data["abbrev"] ## because [] returns a list so
```

```
$abbrev
```

```
[1] "AL" "AK" "AZ" "AR" "CA" "CO" "CT" "DE" "FL" "GA" "HI" "ID" "IL" "IN" "IA"  
[16] "KS" "KY" "LA" "ME" "MD" "MA" "MI" "MN" "MS" "MO" "MT" "NE" "NV" "NH" "NJ"  
[31] "NM" "NY" "NC" "ND" "OH" "OK" "OR" "PA" "RI" "SC" "SD" "TN" "TX" "UT" "VT"  
[46] "VA" "WA" "WV" "WI" "WY"
```

```
state_data[c(2, 7)] ## works, the $ and the [[]] cannot do this
```

```
$abbrev
```

```
[1] "AL" "AK" "AZ" "AR" "CA" "CO" "CT" "DE" "FL" "GA" "HI" "ID" "IL" "IN" "IA"  
[16] "KS" "KY" "LA" "ME" "MD" "MA" "MI" "MN" "MS" "MO" "MT" "NE" "NV" "NH" "NJ"  
[31] "NM" "NY" "NC" "ND" "OH" "OK" "OR" "PA" "RI" "SC" "SD" "TN" "TX" "UT" "VT"  
[46] "VA" "WA" "WV" "WI" "WY"
```

```
$area
```

```
[1] 51609 589757 113909 53104 158693 104247 5009 2057 58560 58876  
[11] 6450 83557 56400 36291 56290 82264 40395 48523 33215 10577  
[21] 8257 58216 84068 47716 69686 147138 77227 110540 9304 7836  
[31] 121666 49576 52586 70665 41222 69919 96981 45333 1214 31055  
[41] 77047 42244 267339 84916 9609 40815 68192 24181 56154 97914
```

```
state_data[c("abbrev", "area")] ## also works same as the previous line
```

```
$abbrev
```

```
[1] "AL" "AK" "AZ" "AR" "CA" "CO" "CT" "DE" "FL" "GA" "HI" "ID" "IL" "IN" "IA"  
[16] "KS" "KY" "LA" "ME" "MD" "MA" "MI" "MN" "MS" "MO" "MT" "NE" "NV" "NH" "NJ"  
[31] "NM" "NY" "NC" "ND" "OH" "OK" "OR" "PA" "RI" "SC" "SD" "TN" "TX" "UT" "VT"  
[46] "VA" "WA" "WV" "WI" "WY"
```

```
$area
```

```
[1] 51609 589757 113909 53104 158693 104247 5009 2057 58560 58876  
[11] 6450 83557 56400 36291 56290 82264 40395 48523 33215 10577  
[21] 8257 58216 84068 47716 69686 147138 77227 110540 9304 7836  
[31] 121666 49576 52586 70665 41222 69919 96981 45333 1214 31055  
[41] 77047 42244 267339 84916 9609 40815 68192 24181 56154 97914
```

```
state_data[[2]] ## the double square bracket structure -- different
```

```
[1] "AL" "AK" "AZ" "AR" "CA" "CO" "CT" "DE" "FL" "GA" "HI" "ID" "IL" "IN" "IA"  
[16] "KS" "KY" "LA" "ME" "MD" "MA" "MI" "MN" "MS" "MO" "MT" "NE" "NV" "NH" "NJ"  
[31] "NM" "NY" "NC" "ND" "OH" "OK" "OR" "PA" "RI" "SC" "SD" "TN" "TX" "UT" "VT"  
[46] "VA" "WA" "WV" "WI" "WY"
```

```
state_data[["abbrev"]] ## returns a vector, not a list any more
```

```
[1] "AL" "AK" "AZ" "AR" "CA" "CO" "CT" "DE" "FL" "GA" "HI" "ID" "IL" "IN" "IA"  
[16] "KS" "KY" "LA" "ME" "MD" "MA" "MI" "MN" "MS" "MO" "MT" "NE" "NV" "NH" "NJ"  
[31] "NM" "NY" "NC" "ND" "OH" "OK" "OR" "PA" "RI" "SC" "SD" "TN" "TX" "UT" "VT"  
[46] "VA" "WA" "WV" "WI" "WY"
```

Study the results

Note the differences.

```
is(state_data$abbrev)
```

```
[1] "character"          "vector"              "data.frameRowLabels"  
[4] "SuperClassMethod"
```

```
is(state_data[2])
```

```
[1] "list"    "vector"
```



```
is(state_data["abbrev"])
```

```
[1] "list"    "vector"
```

```
is(state_data[c(2, 7)])
```

```
[1] "list"    "vector"
```

```
is(state_data[c("abbrev", "area")])
```

```
[1] "list"    "vector"
```

```
is(state_data[[2]])
```

```
[1] "character"          "vector"          "data.frameRowLabels"  
[4] "SuperClassMethod"
```

```
is(state_data[["abbrev"]])
```

```
[1] "character"          "vector"          "data.frameRowLabels"  
[4] "SuperClassMethod"
```

Different question – extract “CA”

- A test of your understanding of extraction

```
state_data$abbrev[5]
```

```
[1] "CA"
```

```
state_data[2][[1]][5] ## ask why?
```

```
[1] "CA"
```

```
state_data[[2]][5]
```

```
[1] "CA"
```

- What about “CA” and “NV”? (think sequences and extract by position)

```
state_data$abbrev[c(5, 28)]
```

```
[1] "CA" "NV"
```

```
state_data[2][[1]][c(5,28)]
```

```
[1] "CA" "NV"
```

```
state_data[[2]][c(5, 28)]
```

```
[1] "CA" "NV"
```

Must be named

- If we try to use names such as “CA” and “NV” instead of 5, 28 – our code will throw an error
- If you have named vector, extraction by name or index number will work:

```
state_data$euro
```

	ATS	BEF	DEM	ESP	FIM	FRF
	13.760300	40.339900	1.955830	166.386000	5.945730	6.559570
	IEP	ITL	LUF	NLG	PTE	
	0.787564	1936.270000	40.339900	2.203710	200.482000	

```
state_data$euro[c("FRF", "DEM")]
```

	FRF	DEM
	6.55957	1.95583

```
state_data[["euro"]]
```

	ATS	BEF	DEM	ESP	FIM	FRF
	13.760300	40.339900	1.955830	166.386000	5.945730	6.559570
	IEP	ITL	LUF	NLG	PTE	
	0.787564	1936.270000	40.339900	2.203710	200.482000	

```
state_data[["euro"]][c("FRF", "DEM")]
```

	FRF	DEM
	6.55957	1.95583

Why do they do this? Why lists?

- If a programmer wants a single long and complex structure and also wants each piece of it to keep its original type, they use a list to contain all of the pieces.
- Recall vectors (atomic ones) coerce the data to keep the type with the highest priority (e.g., logical coerces to integer, integer to numeric, numeric to character etc.)
- Many important model objects are lists, like the results of a regression and more advanced models.
- Many heavily used functions produce lists

Example

```
model1 <- lm(mpg ~ wt, data = mtcars) # this is ols (ordinary least squares) in R
is.list(model1)
```

```
[1] TRUE
```

- usually the output of most modeling functions is a list because output is complicated (a mixture of vectors of different type and length)

```
is.list(summary(model1))
```

```
[1] TRUE
```

```
summary(model1) # what does it look like
```

```

Call:
lm(formula = mpg ~ wt, data = mtcars)

Residuals:
    Min       1Q   Median       3Q      Max
-4.5432 -2.3647 -0.1252  1.4096  6.8727

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  37.2851     1.8776   19.858 < 2e-16 ***
wt          -5.3445     0.5591   -9.559 1.29e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.046 on 30 degrees of freedom
Multiple R-squared:  0.7528,    Adjusted R-squared:  0.7446
F-statistic: 91.38 on 1 and 30 DF,  p-value: 1.294e-10

```

Example(cont'd)

- We can extract the specific output from a complex model by understanding lists
- This becomes very important when we want to quantify model stability by examining thousands of resampled or “bootstrapped” (101A, 101C) models – we would extract the coefficients from each model.
- We see this in machine/statistical learning (101C)

```
names(model1)
```

```

[1] "coefficients" "residuals"    "effects"      "rank"
[5] "fitted.values" "assign"       "qr"           "df.residual"
[9] "xlevels"      "call"        "terms"        "model"

```

```
str(model1) # do you think you could extract the coefficients from our model?
```

List of 12

```
$ coefficients : Named num [1:2] 37.29 -5.34
..- attr(*, "names")= chr [1:2] "(Intercept)" "wt"
$ residuals    : Named num [1:32] -2.28 -0.92 -2.09 1.3 -0.2 ...
..- attr(*, "names")= chr [1:32] "Mazda RX4" "Mazda RX4 Wag" "Datsun 710" "Hornet 4 Drive" ...
$ effects      : Named num [1:32] -113.65 -29.116 -1.661 1.631 0.111 ...
..- attr(*, "names")= chr [1:32] "(Intercept)" "wt" "" "" ...
$ rank         : int 2
$ fitted.values: Named num [1:32] 23.3 21.9 24.9 20.1 18.9 ...
..- attr(*, "names")= chr [1:32] "Mazda RX4" "Mazda RX4 Wag" "Datsun 710" "Hornet 4 Drive" ...
$ assign       : int [1:2] 0 1
$ qr           :List of 5
..$ qr        : num [1:32, 1:2] -5.657 0.177 0.177 0.177 0.177 ...
.. ..- attr(*, "dimnames")=List of 2
.. .. ..$ : chr [1:32] "Mazda RX4" "Mazda RX4 Wag" "Datsun 710" "Hornet 4 Drive" ...
.. .. ..$ : chr [1:2] "(Intercept)" "wt"
.. ..- attr(*, "assign")= int [1:2] 0 1
..$ qraux: num [1:2] 1.18 1.05
..$ pivot: int [1:2] 1 2
..$ tol   : num 1e-07
..$ rank  : int 2
..- attr(*, "class")= chr "qr"
$ df.residual : int 30
$ xlevels      : Named list()
$ call         : language lm(formula = mpg ~ wt, data = mtcars)
$ terms       :Classes 'terms', 'formula' language mpg ~ wt
.. ..- attr(*, "variables")= language list(mpg, wt)
.. ..- attr(*, "factors")= int [1:2, 1] 0 1
.. .. ..- attr(*, "dimnames")=List of 2
```

```

.. ..$ : chr [1:2] "mpg" "wt"
.. ..$ : chr "wt"
.. ..- attr(*, "term.labels")= chr "wt"
.. ..- attr(*, "order")= int 1
.. ..- attr(*, "intercept")= int 1
.. ..- attr(*, "response")= int 1
.. ..- attr(*, ".Environment")=<environment: R_GlobalEnv>
.. ..- attr(*, "predvars")= language list(mpg, wt)
.. ..- attr(*, "dataClasses")= Named chr [1:2] "numeric" "numeric"
.. ..- attr(*, "names")= chr [1:2] "mpg" "wt"
$ model      : 'data.frame': 32 obs. of 2 variables:
..$ mpg: num [1:32] 21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
..$ wt : num [1:32] 2.62 2.88 2.32 3.21 3.44 ...
..- attr(*, "terms")=Classes 'terms', 'formula' language mpg ~ wt
.. ..- attr(*, "variables")= language list(mpg, wt)
.. ..- attr(*, "factors")= int [1:2, 1] 0 1
.. ..- attr(*, "dimnames")=List of 2
.. ..$ : chr [1:2] "mpg" "wt"
.. ..$ : chr "wt"
.. ..- attr(*, "term.labels")= chr "wt"
.. ..- attr(*, "order")= int 1
.. ..- attr(*, "intercept")= int 1
.. ..- attr(*, "response")= int 1
.. ..- attr(*, ".Environment")=<environment: R_GlobalEnv>
.. ..- attr(*, "predvars")= language list(mpg, wt)
.. ..- attr(*, "dataClasses")= Named chr [1:2] "numeric" "numeric"
.. ..- attr(*, "names")= chr [1:2] "mpg" "wt"
- attr(*, "class")= chr "lm"

```

Suggestions

- Practice constructing sequences using the colon operator (:) and seq(). Those work on numeric data and can increment and decrement.
- Practice constructing any type (logical, character, numeric) using concatenate/combine with c() or even rep()
- Practice on simple vectors such as rivers or extract a vector from iris (e.g., iris\$Sepal.Width) and then extract specific values
- Practice extracting from entire lists, for example, take a subset of rows or of column or both. Then parts of rows, parts of columns.
- Work your way up to the list.
- Extraction is a bit like solving a puzzle.

Attendance Opportunity

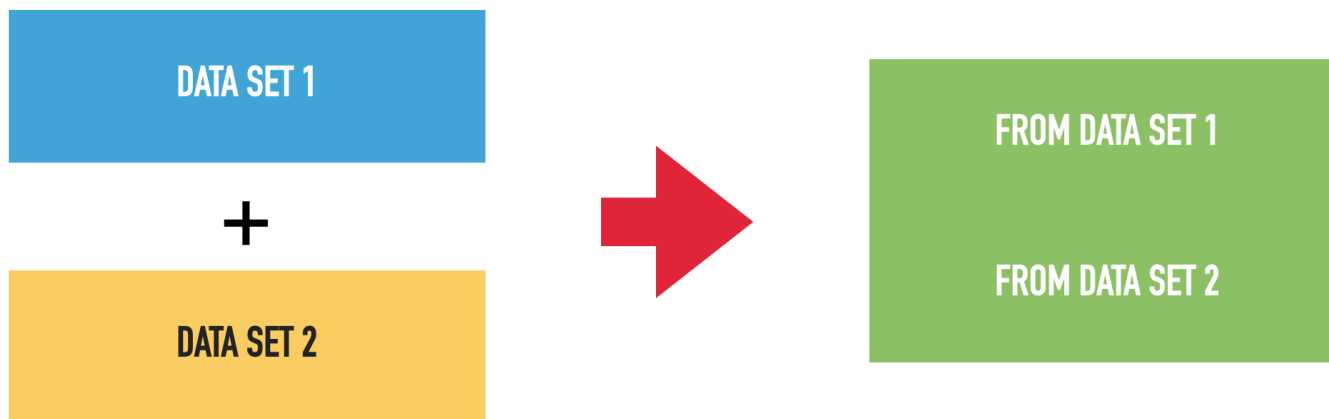
Activity #2: Please Greet Your Neighbor(s)

- You are experts at greeting your neighbors now.
- You need at least one different neighbor each week for your attendance credit (only one photo required)
- You cannot receive credit if it's only you in the photo
- [Social Isolation and Loneliness are public health problems](#)
- Meet a new classmate, take a photograph.

Working with Multiple Data Files

Appending - Adding Rows - Binding Rows

A common task – increasing the number of rows in matrix or a data frame.



The context, the why?

- You might have data from multiple time periods (months, years, etc.)
- You wish to combine them into one larger single file for analysis.
- Example: someone has given you 3 csv files for 3 different time periods
- The columns are all the same, but the rows are different and the data covers different time periods

- The data is from [Box Office Mojo](#), a movie website
- We take the web locations and save them as an R object

```
bom00_09_url <- "https://raw.githubusercontent.com/lewv/data-for-example/refs/heads/main/BOM00_09.csv"
bom10_23_url <- "https://raw.githubusercontent.com/lewv/data-for-example/refs/heads/main/BOM10_23.csv"
bom24_url <- "https://raw.githubusercontent.com/lewv/data-for-example/refs/heads/main/BOM24.csv"
```

The data - 3 files

```
BOM00_09 <- read.csv(bom00_09_url)
names(BOM00_09)
```

```
[1] "Rank"           "Release.Group"  "Worldwide"      "Domestic"
[5] "Domestic_percent" "Foreign"        "Foreign_percent" "year"
```

```
dim(BOM00_09)
```

```
[1] 2000    8
```

```
head(BOM00_09)
```

	Rank	Release.Group	Worldwide	Domestic
1	0	Mission: Impossible II	54,63,88,108	21,54,09,889
2	1	Gladiator	46,05,83,960	18,77,05,427
3	2	Cast Away	42,96,32,142	23,36,32,142
4	3	What Women Want	37,41,11,707	18,28,11,707
5	4	Dinosaur	34,98,22,765	13,77,48,063

```

6      5 How the Grinch Stole Christmas 34,58,42,198 26,07,45,620
      Domestic_percent      Foreign Foreign_percent year
1      39.40% 33,09,78,219      60.60% 2000
2      40.80% 27,28,78,533      59.20% 2000
3      54.40% 19,60,00,000      45.60% 2000
4      48.90% 19,13,00,000      51.10% 2000
5      39.40% 21,20,74,702      60.60% 2000
6      75.40% 8,50,96,578      24.60% 2000

```

```

BOM10_23 <- read.csv(bom10_23_url)
names(BOM10_23)

```

```

[1] "Rank"          "Release.Group"  "Worldwide"      "Domestic"
[5] "Domestic_percent" "Foreign"        "Foreign_percent" "year"

```

```

dim(BOM10_23)

```

```

[1] 2800      8

```

```

head(BOM10_23)

```

```

      Rank      Release.Group      Worldwide      Domestic
1      0      Toy Story 3 1,06,69,69,703 41,50,04,880
2      1      Alice in Wonderland 1,02,54,67,110 33,41,91,110
3      2 Harry Potter and the Deathly Hallows: Part 1 96,02,83,305 29,59,83,305
4      3      Inception 82,82,58,695 29,25,76,195
5      4      Shrek Forever After 75,26,00,867 23,87,36,787
6      5      The Twilight Saga: Eclipse 69,84,91,347 30,05,31,751
      Domestic_percent      Foreign Foreign_percent year

```

1	38.90%	65,19,64,823	61.10%	2010
2	32.60%	69,12,76,000	67.40%	2010
3	30.80%	66,43,00,000	69.20%	2010
4	35.30%	53,56,82,500	64.70%	2010
5	31.70%	51,38,64,080	68.30%	2010
6	43%	39,79,59,596	57%	2010

```
BOM24 <- read.csv(bom24_url)
names(BOM24)
```

```
[1] "Rank"          "Release.Group"  "Worldwide"      "Domestic"
[5] "Domestic_percent" "Foreign"        "Foreign_percent" "year"
```

```
dim(BOM24)
```

```
[1] 200  8
```

```
head(BOM24)
```

	Rank		Release.Group		Worldwide		Domestic
1	0		Inside Out 2	1,59,59,83,694	63,78,52,840		
2	1		Deadpool & Wolverine	1,04,31,80,185	50,69,37,007		
3	2		Despicable Me 4	80,85,37,571	33,25,83,715		
4	3		Dune: Part Two	71,18,44,358	28,21,44,358		
5	4	Godzilla x Kong: The New Empire		56,76,50,016	19,63,50,016		
6	5	Kung Fu Panda 4		54,78,20,030	19,35,90,620		
		Domestic_percent	Foreign	Foreign_percent	year		
1		40%	95,81,30,854	60%	2024		
2		48.60%	53,62,43,178	51.40%	2024		

3	41.10%	47,59,53,856	58.90%	2024
4	39.60%	42,97,00,000	60.40%	2024
5	34.60%	37,13,00,000	65.40%	2024
6	35.30%	35,42,29,410	64.70%	2024

rbind()

Short for “row bind”, implies combining or “binding” data on the row dimension

```
BOM0024 <- rbind(BOM00_09, BOM10_23, BOM24)
names(BOM0024)
```

```
[1] "Rank"           "Release.Group"   "Worldwide"       "Domestic"
[5] "Domestic_percent" "Foreign"         "Foreign_percent" "year"
```

```
dim(BOM0024)
```

```
[1] 5000    8
```

```
head(BOM0024)
```

	Rank		Release.Group	Worldwide	Domestic
1	0	Mission: Impossible II	54,63,88,108	21,54,09,889	
2	1	Gladiator	46,05,83,960	18,77,05,427	
3	2	Cast Away	42,96,32,142	23,36,32,142	
4	3	What Women Want	37,41,11,707	18,28,11,707	
5	4	Dinosaur	34,98,22,765	13,77,48,063	
6	5	How the Grinch Stole Christmas	34,58,42,198	26,07,45,620	
		Domestic_percent	Foreign	Foreign_percent	year

1	39.40%	33,09,78,219	60.60%	2000
2	40.80%	27,28,78,533	59.20%	2000
3	54.40%	19,60,00,000	45.60%	2000
4	48.90%	19,13,00,000	51.10%	2000
5	39.40%	21,20,74,702	60.60%	2000
6	75.40%	8,50,96,578	24.60%	2000

```
tail(BOM0024)
```

	Rank	Release.Group	Worldwide	Domestic
4995	194	Jesus Thirsts: The Miracle of the Eucharist	29,27,121	29,27,121
4996	195	Buzz House: The Movie	29,10,112	0
4997	196	We 12	28,64,154	0
4998	197	Me Contro Te - Il film: Operazione Spie	27,98,501	0
4999	198	Menudas piezas	27,78,043	0
5000	199	Silent Love	27,39,320	0

	Domestic_percent	Foreign	Foreign_percent	year
4995	100%	0	0	2024
4996	0	29,10,112	100%	2024
4997	0	28,64,154	100%	2024
4998	0	27,98,501	100%	2024
4999	0	27,78,043	100%	2024
5000	0	27,39,320	100%	2024

Result

- can anyone tell me why only a few of the variables are numeric?

```
str(BOM0024)
```

```
'data.frame': 5000 obs. of 8 variables:
 $ Rank      : int  0 1 2 3 4 5 6 7 8 9 ...
 $ Release.Group : chr  "Mission: Impossible II" "Gladiator" "Cast Away" "What Women Want" ...
 $ Worldwide    : chr  "54,63,88,108" "46,05,83,960" "42,96,32,142" "37,41,11,707" ...
 $ Domestic     : chr  "21,54,09,889" "18,77,05,427" "23,36,32,142" "18,28,11,707" ...
 $ Domestic_percent: chr  "39.40%" "40.80%" "54.40%" "48.90%" ...
 $ Foreign      : chr  "33,09,78,219" "27,28,78,533" "19,60,00,000" "19,13,00,000" ...
 $ Foreign_percent : chr  "60.60%" "59.20%" "45.60%" "51.10%" ...
 $ year         : int  2000 2000 2000 2000 2000 2000 2000 2000 2000 2000 2000 ...
```

Solution (Week 3)

- Remove non-numeric characters first, then convert to numeric.
- We could remove the characters (like the comma and percentage) in this manner
- gsub is a “global substitution” function, it searches for the character we specify and then removes all occurrences.
- might be easiest understood reading from the innermost part of the R function

```
# for commas |> is a "pipe"
# The pipe |> means:
# "Take the result on the left and pass it into the function on the right."

BOM0024$Worldwide <- gsub(",", "", BOM0024$Worldwide) |> as.numeric()
BOM0024$Domestic <- gsub(",", "", BOM0024$Domestic) |> as.numeric()
BOM0024$Foreign <- gsub(",", "", BOM0024$Foreign) |> as.numeric()

# for percent we nested it instead to show the readability of |>
BOM0024$Domestic_percent <- (gsub("%", "", BOM0024$Domestic_percent) |> as.numeric())/100
```

Warning: NAs introduced by coercion

```
BOM0024$Foreign_percent <- (gsub("%", "", BOM0024$Foreign_percent) |> as.numeric())/100
```

Warning: NAs introduced by coercion

Substitution result

- Always check your work!

```
str(BOM0024)
```

```
'data.frame': 5000 obs. of 8 variables:
 $ Rank      : int  0 1 2 3 4 5 6 7 8 9 ...
 $ Release.Group : chr  "Mission: Impossible II" "Gladiator" "Cast Away" "What Women Want" ...
 $ Worldwide    : num  5.46e+08 4.61e+08 4.30e+08 3.74e+08 3.50e+08 ...
 $ Domestic     : num  2.15e+08 1.88e+08 2.34e+08 1.83e+08 1.38e+08 ...
 $ Domestic_percent: num  0.394 0.408 0.544 0.489 0.394 0.754 0.503 0.556 0.531 0.533 ...
 $ Foreign      : num  3.31e+08 2.73e+08 1.96e+08 1.91e+08 2.12e+08 ...
 $ Foreign_percent : num  0.606 0.592 0.456 0.511 0.606 0.246 0.497 0.444 0.469 0.467 ...
 $ year        : int  2000 2000 2000 2000 2000 2000 2000 2000 2000 2000 ...
```

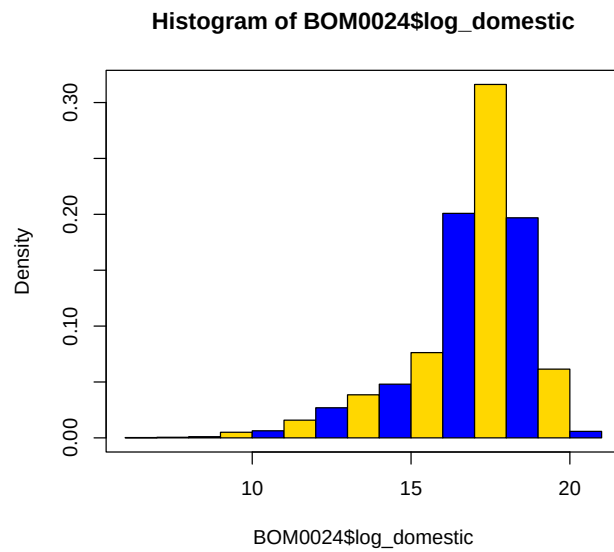
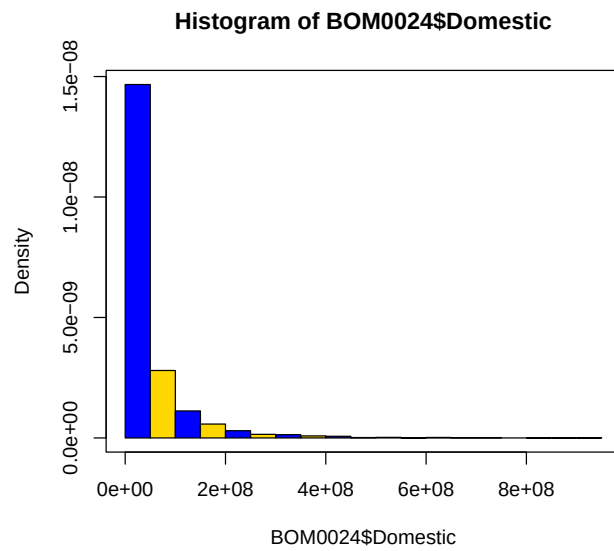
Looking ahead to next week

- Probably needs more work but... a glimpse of next week
- We log to normalize the variable with a long right tail
- `hist()` will plot a histogram of a numeric variable
- the rest is about beauty


```
log_domestic <- log(BOM0024$Domestic)

par(mfrow = c(2,1)) # change default to 2 rows of plots, one column
hist(BOM0024$Domestic, freq = FALSE, col = c("blue", "gold"))
hist(log_domestic, freq = FALSE, col = c("blue", "gold"))
box()
par(mfrow = c(1,1)) # change back to a single plot
```


Result



Joining/Merging

- This a fundamental data handling skill which borders on intermediate
- As in the previous example, data are broken up into separate files. BUT the information in the columns is different each file and the rows are the same (or mostly the same)
- This is a situation where one is trying to augment data on existing observations with additional data on the column dimension (added characteristics, added variables).
- Generally, you should want a “key” or an identifier variable/column that you can match your datasets on before doing this.
- We are going to get some data for this one

A little webscraping

- GET SOME (FREE) DATA FROM THE WEB!
- Maybe violate EULAs and other things (so you don't need to run this part)
- But it's easy in R (BTW Python is the preferred tool here)
- You will to `install.packages("rvest")` and then `load(rvest)`
- This one is easy, we are going to grab some tables of US data (acquisition)
- Maybe clean them up a little (preparation)
- Save them as R Data Frames
- Then join them to make a more complete dataset

A Wikipedia page on the US



Figure 1: millionaire stats

Another Wikipedia page on the US

Contents

hide

(Top)

List

See also

References

List of U.S. states by research and development spending

1 language

Article

Talk

Read

Edit

View history

Tools

From Wikipedia, the free encyclopedia

This is a list of **U.S. states** and the **District of Columbia** by **research and development (R&D) spending** in 2020 adjusted **US dollar**.

List

[edit]

States by R&D spending, spending per capita and federal government spending as percentage of total R&D spending.

Science and engineering in the USA by state, 2010

Three states fall into the top category in both maps: Maryland, Massachusetts and Washington

Science and engineering occupations as a share of all occupations, 2010 (%)

The mean is 6.5%

5.0% and above

4.0% - 5.0%

3.0% - 4.0%

2.0% - 3.0%

Below 2.0%

32%

National contribution of seven states with the highest share of science and engineering occupations (in descending order): District of Columbia, Virginia, Washington, Maryland, Massachusetts, Colorado and California

13.7%

California's share of national science and engineering occupations, the top US state for this indicator

R&D performed as a share of state GDP, 2010 (%)

The mean is 2.5%

3.0% and above

2.0% - 3.0%

1.0% - 2.0%

Below 1.0%

42%

Contribution of six states to national R&D expenditure: New Mexico, Maryland, Washington, Massachusetts, California and Michigan

R&D employment and spending (2010)

U.S. states by R&D spending 2020 (in adjusted 2020 dollars)

National rank	State	Expenditures on R&D (millions of US\$) ^[1]	Expenditures on R&D per capita in US\$ ^[2]	Federal government share in % ^[3]	Expenditures on R&D in % of GDP ^[1]
1	 California	217,976	4,220	1.6	7.05%
2	 Washington	46,392	4,496	0.6	7.50%
3	 Massachusetts	44,907	5,188	1.3	7.69%

Figure 2: research & development

32

Activate library, create constants

```
library(rvest)

millionaires_page <- "https://en.wikipedia.org/wiki/List_of_U.S._states_by_the_number_of_millionaire_households#List"

rd_page <- "https://en.wikipedia.org/wiki/List_of_U.S._states_by_research_and_development_spending#List"
```

See rvest.tidyverse.org more explanations

- The rvest library has a little function `read_html()` basically reads the web location you give it.
- Typically, when scraping Wikipedia, you want their tables, so we search for tables

```
millionaires <- read_html(millionaires_page)

millionaires |> html_elements("table")
```

```
{xml_nodeset (2)}
[1] <table class="wikitable nowrap sortable">\n<caption>States by number and ...
[2] <table class="nowraplinks hlist mw-collapsible mw-collapsed navbox-inner" ...
```

- It turns out to be the first element

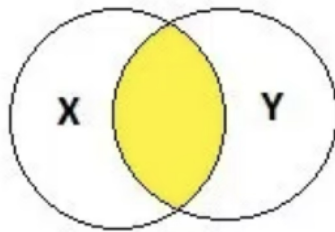
Extract the data table with `html_table()`

- We decide to extract the first table. Do the same for `r_d`.
- And check the results, these are data frames

```
millionaires <- (millionaires_page |> read_html() |> html_table()[[1]])  
  
r_d <- (rd_page |> read_html() |> html_table()[[1]])
```

Join/Merge two data frames

- The next slide shows a merge in base R, I will show you a “join” in a different package later.
- This is the simplest type, the intersection of two datasets matched on a specific row value(s), typically an ID (state)
- By convention, X (left, first dataset) and Y (right, second dataset)
- We want the intersection of the two
- In our particular example, States and their measurements (why would we care to do such a thing?)



Code

```
rich_n_research <- merge(millionaires, r_d, by = "State")  
str(rich_n_research)
```

```
'data.frame':  51 obs. of  9 variables:  
 $ State      : chr  "Alabama" "Alaska" "Arizona" "Arkansas" ...
```

```

$ Rank : int 26 47 18 34 1 16 23 45 43 4 ...
$ Number of millionaire households : chr "94,259" "22,302" "161,014" "51,532" ...
$ Share of millionaire households : chr "4.87%" "8.18%" "6.03%" "4.33%" ...
$ Nationalrank : chr "26" "50" "20" "42" ...
$ Expenditures on R&D (millions of US$)[1]: chr "5,701" "339" "9,056" "892" ...
$ Expenditures on R&D per capita in US$[2]: chr "1,006" "410" "1,079" "286" ...
$ Federal government share in %[3] : num 25.3 25.7 2.9 4.6 1.6 6.1 0.2 0.1 58.3 6.4 ...
$ Expenditures on R&D in % of GDP[1] : chr "2.54%" "0.67%" "2.43%" "0.69%" ...

```

```
head(rich_n_research)
```

```

      State Rank Number of millionaire households
1   Alabama  26                94,259
2    Alaska  47                22,302
3   Arizona  18             161,014
4  Arkansas  34                51,532
5 California  1           1,147,251
6   Colorado 16             170,223
Share of millionaire households Nationalrank
1                4.87%                26
2                8.18%                50
3                6.03%                20
4                4.33%                42
5                8.51%                 1
6                7.48%                18
Expenditures on R&D (millions of US$)[1]
1                5,701
2                 339
3                9,056
4                 892
5             217,976

```

6	10,297	
	Expenditures on R&D per capita in US\$[2]	Federal government share in %[3]
1	1,006	25.3
2	410	25.7
3	1,079	2.9
4	286	4.6
5	4,220	1.6
6	1,356	6.1
	Expenditures on R&D in % of GDP[1]	
1	2.54%	
2	0.67%	
3	2.43%	
4	0.69%	
5	7.05%	
6	2.64%	

Need to “clean” the data

- More of this next week but to give you an idea, we are going to create nicely named variables from the old mess ones and clean up the commas
- Always check

```
rich_n_research$expenditures_rd <- as.numeric(gsub(",", "",
  rich_n_research$`Expenditures on R&D (millions of US$)[1]`))
rich_n_research$millionaire_households <- as.numeric(gsub(",", "",
  rich_n_research$`Number of millionaire households`))

summary(rich_n_research)
```

State	Rank	Number of millionaire households
Length:51	Min. : 1.0	Length:51
Class :character	1st Qu.:13.5	Class :character
Mode :character	Median :26.0	Mode :character
	Mean :26.0	
	3rd Qu.:38.5	
	Max. :51.0	
Share of millionaire households	Nationalrank	
Length:51		Length:51
Class :character		Class :character
Mode :character		Mode :character

Expenditures on R&D (millions of US\$)[1]
Length:51
Class :character
Mode :character

Expenditures on R&D per capita in US\$[2]	Federal government share in %[3]
Length:51	Min. : 0.100
Class :character	1st Qu.: 1.400
Mode :character	Median : 2.600
	Mean : 7.516
	3rd Qu.: 9.000
	Max. :58.300

Expenditures on R&D in % of GDP[1]	expenditures_rd	millionaire_households
Length:51	Min. : 339	Min. : 12849
Class :character	1st Qu.: 1472	1st Qu.: 42416

Mode	:character	Median	: 5701	Median	: 94259
		Mean	: 13957	Mean	: 164441
		3rd Qu.	: 12602	3rd Qu.	: 228604
		Max.	:217976	Max.	:1147251

Some stats and modeling

- We are heading towards Stats 101A but...

```
cor(rich_n_research$expenditures_rd, rich_n_research$millionaire_households)
```

```
[1] 0.8521751
```

```
rd_rich_model <- lm(millionaire_households ~ expenditures_rd,
                    data = rich_n_research)

summary(rd_rich_model)
```

Call:

```
lm(formula = millionaire_households ~ expenditures_rd, data = rich_n_research)
```

Residuals:

Min	1Q	Median	3Q	Max
-128186	-67975	-31506	19937	373420

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	8.844e+04	1.626e+04	5.44	1.68e-06 ***

```
expenditures_rd 5.446e+00 4.777e-01 11.40 2.18e-15 ***
```

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 105900 on 49 degrees of freedom
```

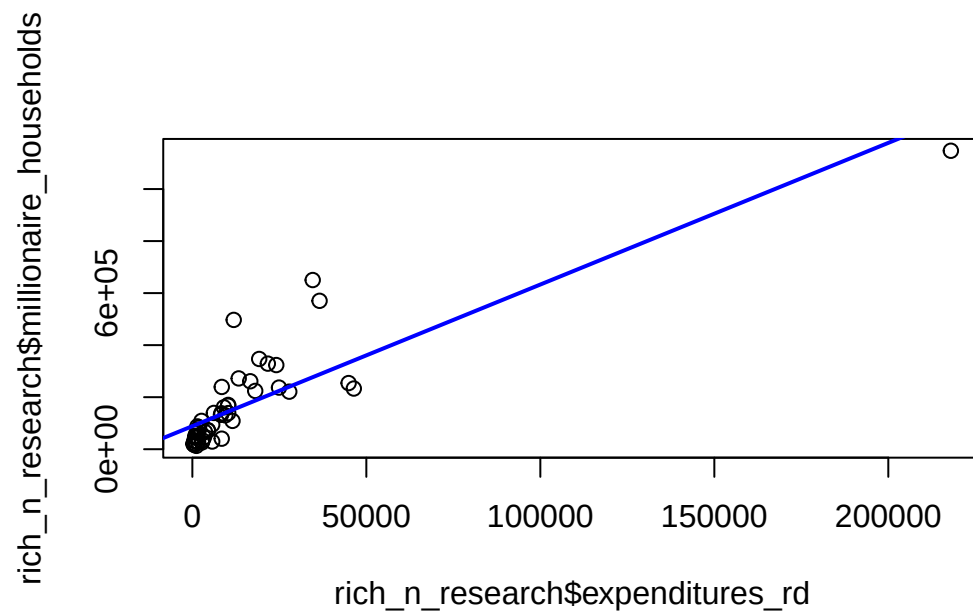
```
Multiple R-squared:  0.7262,    Adjusted R-squared:  0.7206
```

```
F-statistic:   130 on 1 and 49 DF,  p-value: 2.178e-15
```

BUT...

- Why we try to visualize our data

```
plot(rich_n_research$expenditures_rd, rich_n_research$millionaire_households)
abline(rd_rich_model, col = "blue", lwd = 2)
```



- Clearly an outlier

Address the outlier

- I am going to remove the outlier and then model again without it

```
no_california <- rich_n_research[which(!rich_n_research$State %in% c("California")), ]
cor(no_california$expenditures_rd, no_california$millionaire_households)
```

```
[1] 0.7634
```



```
no_ca_rd_rich_model <- lm(millionaire_households ~ expenditures_rd,
                          data = no_california)

summary(no_ca_rd_rich_model)
```

Call:

```
lm(formula = millionaire_households ~ expenditures_rd, data = no_california)
```

Residuals:

Min	1Q	Median	3Q	Max
-256309	-37791	-12882	22082	333196

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	51562.491	17525.503	2.942	0.00501 **
expenditures_rd	9.439	1.153	8.188	1.15e-10 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 94210 on 48 degrees of freedom

Multiple R-squared: 0.5828, Adjusted R-squared: 0.5741

F-statistic: 67.05 on 1 and 48 DF, p-value: 1.146e-10

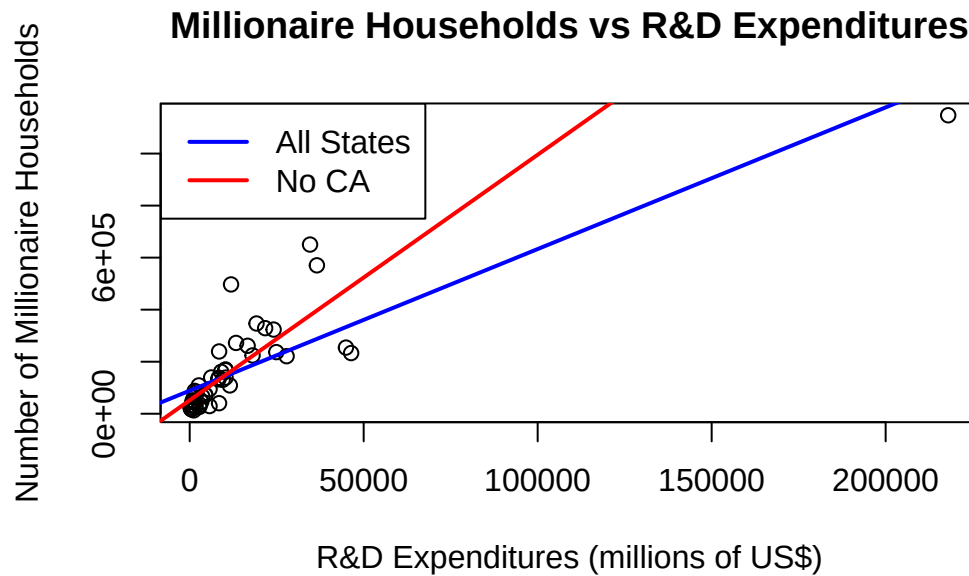
- Leaving California out moved the regression line a lot

One more plot to show the difference

```

plot(x = rich_n_research$expenditures_rd,
     y = rich_n_research$millionaire_households,
     main = "Millionaire Households vs R&D Expenditures",
     xlab = "R&D Expenditures (millions of US$)",
     ylab = "Number of Millionaire Households")
abline(rd_rich_model, col = "blue", lwd = 2)
abline(no_ca_rd_rich_model, col = "red", lwd = 2)
legend("topleft",
      legend = c("All States", "No CA"),
      col = c("blue", "red"),
      lwd = 2,
      bty = "o")

```



Takeaways

- The list, a more complex data structure in R
- Working with multiple data frames in R
 - appending (adding more rows or lengthing the columns)
 - joining (adding more columns or widening the rows)
- Got a peek at a data visualization comparing two histograms
- Threw in web scraping using rvest

Have a peaceful weekend